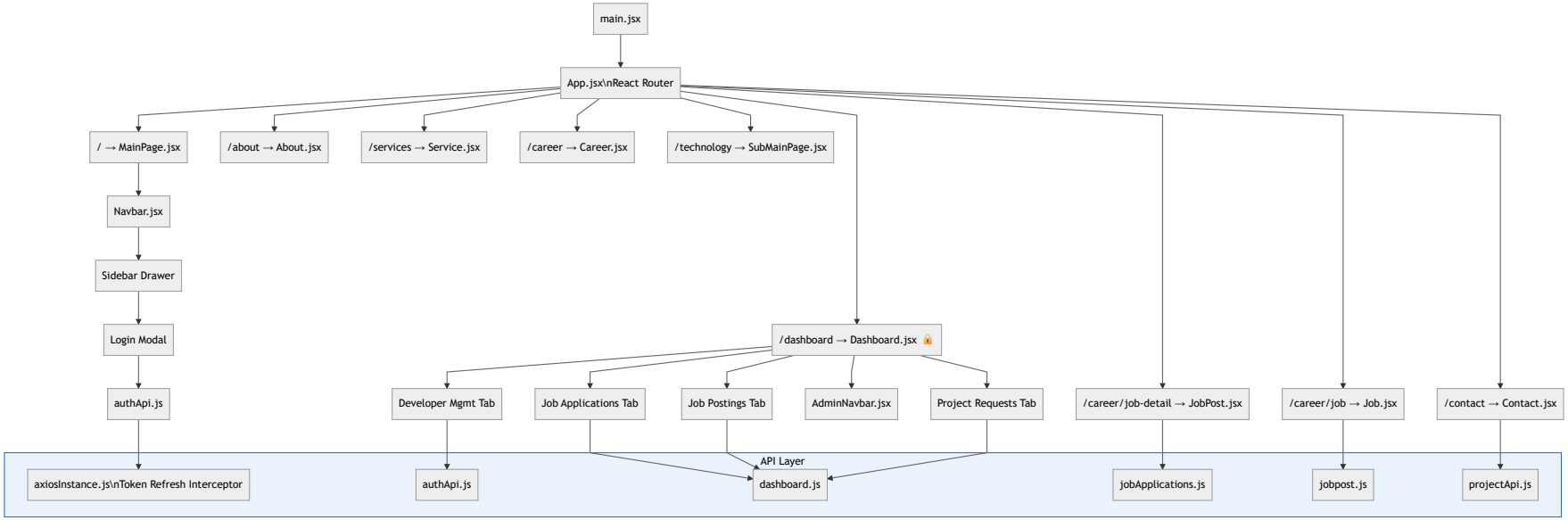
📘 Hously Finttech Realty — Documentation Part 2

Frontend Architecture & Component Reference

---

8. Frontend Component Architecture

Component Tree


main.jsx
App.jsx\nReact Router
/ → MainPage.jsx
/about → About.jsx
/services → Service.jsx
/career → Career.jsx
/technology → SubMainPage.jsx
Navbar.jsx
Sidebar Drawer
Login Modal
authApi.js
/dashboard → Dashboard.jsx 🔒
Developer Mgmt Tab
Job Applications Tab
Job Postings Tab
AdminNavbar.jsx
Project Requests Tab
/career/job-detail → JobPost.jsx
/career/job → Job.jsx
/contact → Contact.jsx
API Layer
axiosInstance.js\nToken Refresh Interceptor
authApi.js
dashboard.js
jobApplications.js
jobpost.js
projectApi.js

9. Component Reference

9.1 `Navbar.jsx` — Navigation + Auth Entry Point

Location: `client/src/components/Navbar.jsx`

Key Features:

Feature	Description
Fixed top navbar	Scrolls with page, transparent → solid on scroll
Nav Links	Home, About, Services, Career, Contact
Right Button	Hamburger / user icon → opens sidebar drawer
Sidebar	Slides in from right; contains logo + nav links
Logo Click	Opens Login Modal popup
Login Modal	Email + Password form → calls `loginApi()`
Session State	Stores user in `sessionStorage("hously_user")`
Post-Login	Shows user name + Logout button in sidebar

Login Flow Code Pattern:

```
// Navbar.jsx - Login handler
const handleLogin = async (e) => {
  e.preventDefault();
  const data = await loginApi(email, password);
  sessionStorage.setItem("hously_user", JSON.stringify(data.user));
  setUser(data.user);
  setShowLoginModal(false);
};
```

9.2 `Home.jsx` — Landing Page

Location: `client/src/components/Home.jsx`
Route: `/` (via `MainPage.jsx`)

Sections:

Section	Description
Hero	Full-screen with animated headline, CTA buttons
About Preview	Company stats + glassmorphism cards
Services	3-column service cards with icons
Why Choose Us	Feature highlights with icons

Section	Description
Testimonials	Swiper.js carousel with client reviews
CTA Section	“Get Started Today” call-to-action

9.3 `About.jsx` — About Page

Route: `/about`

Displays company history, mission/vision, team introduction, and milestones with animated counters and premium card layouts.

9.4 `Service.jsx` — Services Page

Route: `/services`

Lists all professional services offered: Real Estate Advisory, Fintech Solutions, Property Management, Investment Consulting — with detailed descriptions and feature lists.

9.5 `Contact.jsx` — Contact + Project Inquiry

Route: `/contact`

Form Fields:

Field	Type	Required
Full Name	text	✅
Company Name	text	❌
Email	email	✅
Phone	tel	❌
Inquiry Type	select	✅
Service Required	select	✅
Message	textarea	✅

On Submit: `POST /api/projects/create` → stores in `project_requests` table.

9.6 `Career.jsx` — Career Overview

Route: `/career`

Landing page for career section — shows company culture, benefits, and a CTA to view open positions linking to `/career/job`.

9.7 `Job.jsx` — Job Listings

Route: `/career/job`

Features:

Feature	Description
Data Fetch	`GET /api/projects/jobs` on mount
Search	Filter by job title keyword
Type Filter	Full-time / Part-time / Contract
Location Filter	Remote / Hybrid / On-site
Sort	By date, salary
Job Cards	Shows title, company, type, location, salary, isNew badge
Click	Navigates to `/career/job-detail` with job data via state

9.8 `JobPost.jsx` — Job Detail + Apply Form

Route: `/career/job-detail`

Job Detail Panel:

- Title, Company, Type, Location, Salary
- About Company, Description
- Responsibilities list, Requirements list, What You'll Gain list

Apply Form Fields:

Field	Type	Required
Full Name	text	✅
Email	email	✅
Phone	tel	✅
Experience (years)	number	✅
Skills	text	✅
10th Percentage	number	✅
12th Percentage	number	✅
Graduation %	number	✅
College Name	text	✅
Branch	text	✅
Resume	file (PDF/DOC)	✅

On Submit: `POST /api/jobs/apply` with `multipart/form-data` — resume stored as BLOB.

9.9 `Dashboard.jsx` — Admin Dashboard

Route: `/dashboard` (Protected by session check)
Location: `client/src/pages/Dashboard.jsx`

Dashboard

Dashboard Sections:

Tab 1: Job Applications

Action	API Call	Description
Load	`GET /api/jobs/`	Fetch all applications
View Resume	`GET /api/jobs/resume/:id`	Opens PDF/DOC in new tab
Change Status	`PUT /api/jobs/:id/status`	pending/reviewed/selected/rejected
Delete	`DELETE /api/jobs/:id`	Remove application

Status Badges:

- 🟡 `pending` — yellow badge
- 🔵 `reviewed` — blue badge
- 🟢 `selected` — green badge
- 🔴 `rejected` — red badge

Tab 2: Job Postings

Action	API Call	Description
Load	`GET /api/projects/jobs`	Fetch all job posts
Create	`POST /api/projects/create-job`	Add new job opening
Edit	`PUT /api/projects/jobs/:id`	Update existing job
Delete	`DELETE /api/projects/jobs/:id`	Remove job posting

Create/Edit Form includes: title, company, type, location, locationType, salary, description, aboutCompany, responsibilities (array), requirements (array), gains (array).

Tab 3: Project Requests

Action	API Call	Description
Load	`GET /api/projects/get-all`	Fetch all inquiries
Update Status	`PUT /api/projects/:id/status`	Mark as reviewed/closed
Delete	`DELETE /api/projects/:id`	Remove inquiry

Tab 4: Developer Management

Action	API Call	Description
Load Users	`GET /api/auth/users`	List all users
Create Developer	`POST /api/auth/users/create-developer`	Add developer account
Delete User	`DELETE /api/auth/users/:id`	Remove user

9.10 `axiosInstance.js` — HTTP Client with Auto-Refresh

```
Client Request
    |
    ▼
axiosInstance.request()
    |
    ├── Success (2xx) ──────────────────────────→ Return response
    |
    └── Error (401 TOKEN_EXPIRED)
            |
            ├── isRefreshing = true
            ├── POST /api/auth/refresh  ←── httpOnly cookie sent auto
            |       |
            |       ├── Success → processQueue() → Retry original request → ✅
            |       |
            |       └── Failure → sessionStorage.clear() → redirect "/" → ❌
            |
            └── Other requests during refresh → queued in failedQueue[]
```

Config	Value
`baseURL`	`http://localhost:5000`
`withCredentials`	`true` (cookies auto-sent)
`Content-Type`	`application/json`
Auto refresh	On `401 + code: TOKEN_EXPIRED`
Queue handling	Concurrent requests queued during refresh

---

10. Job Application Full Flow

Job Application Flow

```
flowchart TD
    A([User visits /career]) --> B[Career overview + benefits]
    B --> C[Click 'View Open Positions']
    C --> D[/career/job — Job Listings]
    D --> E{Apply Filters}
    E --> F[Filter by: Type / Location / Search]
    F --> G[View Job Cards]
    G --> H[Click Job Card]
    H --> I[/career/job-detail — Job Detail Page]
    I --> J[Read: Description, Requirements, Responsibilities]
    J --> K[Scroll to Apply Form]
    K --> L[Fill Personal Info\nName, Email, Phone]
```

```
    L --> M[Fill Academic Info\n10th, 12th, Graduation %\nCollege, Branch]
    M --> N[Fill Professional Info\nExperience, Skills]
    N --> O[Upload Resume - PDF/DOC]
    O --> P{Form Submit}
    P --> Q[POST /api/jobs/apply\nmultipart/form-data]
    Q --> R{Server Validation}
    R -- Valid --> S[INSERT job_applications\nresume stored as BLOB]
    S --> T[✅ Application Submitted!\nSuccess message shown]
    R -- Invalid --> U[❌ Error shown to user]

    subgraph ADMIN ["Admin Dashboard Flow"]
        V[Admin logs in]
        W[Dashboard → Job Applications Tab]
        X[View all applications table]
        Y{Admin Action}
        Z1[View Resume → stream BLOB as PDF]
        Z2[Update Status → pending/reviewed/selected/rejected]
        Z3[Delete Application]
    end

    T -.-> V
    V --> W --> X --> Y
    Y --> Z1
    Y --> Z2
    Y --> Z3
```

Continued in Part 3: Test Cases, Security, Deployment →